Obliczenia numeryczne w Pythonie biblioteka NumPy

Zestaw ćwiczeń do samodzielnej pracy

Spis treści

1. Wprowadzenie do NumPy 3
1.1. Podstawowe struktury danych 4
1.2. Proste indeksowanie i wycinanie tablic 7
1.3. Podstawowe operacje na danych w tablicach 10
1.4. Funkcje uniwersalne 12
1.5. Broadcasting 14
1.6. Własne funkcje uniwersalne 16
1.7. Funkcje agregujące i statystyki opisowe 18
1.8. Łączenie i dzielenie tablic 23
1.9. Rozszerzanie i powtarzanie tablic 25

1. Wprowadzenie do NumPy

NumPy to fundamentalna biblioteka do obliczeń numerycznych w Pythonie, która wprowadza wydajne struktury danych do manipulacji tablicami wielowymiarowymi. W poniższym zestawie ćwiczeń skupimy się na podstawowych oraz zaawansowanych operacjach na tablicach NumPy, ze szczególnym uwzględnieniem broadcastingu i operacji macierzowych.

Podczas rozwiązywania zadań należy zwrócić szczególną uwagę na różnice między standardowymi operacjami Pythona a operacjami wektoryzowanymi NumPy. Choć często istnieje kilka sposobów osiągnięcia tego samego rezultatu, warto pamiętać, że składnia NumPy została zaprojektowana tak, aby odzwierciedlać matematyczny zapis operacji macierzowych. Właściwe wykorzystanie mechanizmów takich jak broadcasting czy operacje macierzowe (np. operator `@`) nie tylko czyni kod bardziej czytelnym, ale także znacząco wpływa na jego wydajność.

W zadaniach przechodzimy od prostych operacji arytmetycznych na tablicach jednowymiarowych, przez manipulacje kształtem danych, aż po złożone operacje wykorzystujące broadcasting i zaawansowane funkcje biblioteki. Szczególną uwagę należy zwrócić na:

Różnice między operacjami element po elemencie a operacjami macierzowymi
- Właściwe wykorzystanie mechanizmu broadcastingu
- Efektywne przekształcanie wymiarów tablic (`reshape`, `transpose`)
- Unikanie jawnych pętli na rzecz operacji wektoryzowanych

Powodzenia!

Gdzie szukać pomocy:

https://numpy.org/doc/2.1/

1.1. Podstawowe struktury danych

Ćwiczenie 1: Tworzenie jednowymiarowej tablicy
Stwórz jednowymiarową tablicę NumPy zawierającą liczby całkowite od 10 do 20 włącznie. Skorzystaj z funkcji `np.arange()`.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(10, 21)
print(tablica)
```

Ćwiczenie 2: Tworzenie tablicy wielowymiarowej
Utwórz dwuwymiarową tablicę o wymiarach 3x4, wypełnioną liczbami zmiennoprzecinkowymi od 1.0 do 12.0. Możesz użyć funkcji `np.reshape()`.

Rozwiązanie:

```
import numpy as np
liczby = np.arange(1.0, 13.0)
macierz = liczby.reshape(3, 4)
print(macierz)
```

Ćwiczenie 3: Tablica zerowa
Stwórz dwuwymiarową tablicę 5x5 wypełnioną samymi zerami. Wykorzystaj funkcję `np.zeros()`.

Rozwiązanie:

```
import numpy as np
macierz_zer = np.zeros((5, 5))
print(macierz_zer)
```

Ćwiczenie 4: Tablica jednostkowa
Stwórz tablicę jednostkową (macierz jednostkową) o rozmiarze 4x4. Skorzystaj z funkcji `np.eye()`.

Rozwiązanie:

```
import numpy as np
macierz_jedn = np.eye(4)
print(macierz_jedn)
```

Ćwiczenie 5: Tablica wypełniona stałą wartością
Utwórz tablicę **jednowymiarową** z pięcioma elementami, wypełnioną wartością 7. Użyj funkcji `np.full()`.

Rozwiązanie:

```
import numpy as np
siodemki = np.full(5, 7)
print(siodemki)
```

Ćwiczenie 6: Tworzenie tablicy z zakresu liczb
Stwórz tablicę NumPy zawierającą liczby z zakresu od 0 do 1 z krokiem 0.1. Skorzystaj z funkcji `np.linspace()`.

Rozwiązanie:

```
import numpy as np
odcinki = np.linspace(0, 1, 11)
print(odcinki)
```

Ćwiczenie 7: Losowa tablica
Wygeneruj dwuwymiarową tablicę o wymiarach 10x10, wypełnioną losowymi liczbami zmiennoprzecinkowymi z zakresu od 0 do 1. Skorzystaj z funkcji `np.random.random()`.

Rozwiązanie:

```
import numpy as np
losowe = np.random.random((10, 10))
print(losowe)
```

Ćwiczenie 8: Tablica z losowymi liczbami całkowitymi
Utwórz tablicę 6x6 wypełnioną losowymi liczbami całkowitymi z przedziału od 0 do 100. Użyj funkcji `np.random.randint()`.

Rozwiązanie:

```
import numpy as np
losowe_calkowite = np.random.randint(0, 101, size=(6, 6))
print(losowe_calkowite)
```

Ćwiczenie 9: Tablica z wartością początkową, krokiem i wymiarem
Stwórz tablicę jednowymiarową z wartościami zaczynającymi się od 5, z krokiem 2, o długości 10 elementów. Użyj funkcji `np.arange()`.

Rozwiązanie:

```
import numpy as np
seria = np.arange(5, 25, 2)
print(seria)
```

Ćwiczenie 10: Zmienianie kształtu tablicy
Stwórz jednowymiarową tablicę zawierającą liczby od 1 do 16, a następnie przekształć ją w tablicę o wymiarach 4x4. Użyj funkcji `np.reshape()`.

Rozwiązanie:

```
import numpy as np
liczby = np.arange(1, 17)
macierz_4x4 = liczby.reshape(4, 4)
print(macierz_4x4)
```

Ćwiczenie 11: Tworzenie tablicy z określonym typem danych
Stwórz jednowymiarową tablicę NumPy zawierającą liczby całkowite od 1 do 10, ale upewnij się, że typ danych tablicy to `float64`.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(1, 11, dtype=np.float64)
print(tablica)
```

Ćwiczenie 12: Zmiana typu danych istniejącej tablicy
Utwórz tablicę NumPy z liczbami całkowitymi od 10 do 20, a następnie zmień typ danych tej tablicy na `int16`.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(10, 21).astype(np.int16)
```

```
print(tablica)
```

Ćwiczenie 13: Tworzenie tablicy z wartościami zmiennoprzecinkowymi
Stwórz dwuwymiarową tablicę NumPy o wymiarach 4x4 wypełnioną losowymi wartościami zmiennoprzecinkowymi z zakresu od 0 do 1, z typem danych `float32`.

Rozwiązanie:

```
import numpy as np
losowa_macierz = np.random.random((4, 4)).astype(np.float32)
print(losowa_macierz)
```

Ćwiczenie 14: Sprawdzenie typu danych tablicy
Utwórz tablicę NumPy zawierającą liczby od 1 do 5, a następnie sprawdź i wyświetl typ danych tej tablicy.

Rozwiązanie:

```
import numpy as np
tablica = np.array([1, 2, 3, 4, 5])
print("Tablica:", tablica)
print("Typ danych:", tablica.dtype)
```

Ćwiczenie 15: Tworzenie tablicy logicznej
Stwórz tablicę NumPy o wymiarach 3x3 wypełnioną wartościami logicznymi (`True` i `False`), używając typu danych `bool`.

Rozwiązanie:

```
import numpy as np
logiczna = np.array([[True, False, True],
                     [False, True, False],
                     [True, False, True]], dtype=bool)
print(logiczna)
```

1.2. Proste indeksowanie i wycinanie tablic

Ćwiczenie 16: Wyciąganie elementów z tablicy jednowymiarowej
Utwórz jednowymiarową tablicę NumPy zawierającą liczby od 0 do 20. Wyciągnij elementy o indeksach 3, 5 i 7.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(21)
wybrane = tablica[[3, 5, 7]]
print(wybrane)
```

Ćwiczenie 17: Wycinanie fragmentu tablicy dwuwymiarowej
Stwórz dwuwymiarową tablicę o wymiarach 4x4, zawierającą liczby od 1 do 16. Wytnij i wyświetl podtablicę składającą się z wierszy 1 i 2 oraz kolumn 2 i 3.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(1, 17).reshape(4, 4)
wycinek = tablica[1:3, 2:4]
print(tablica)
print(wycinek)
```

Ćwiczenie 18: Wybieranie elementów wiersza
Utwórz dwuwymiarową tablicę 5x5 z liczbami od 0 do 24. Wybierz i wyświetl wszystkie elementy w trzecim wierszu.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(25).reshape(5, 5)
trzeci_wiersz = tablica[2]
print(tablica)
print(trzeci_wiersz)
```

Ćwiczenie 19: Indeksowanie z użyciem zakresu
Stwórz tablicę NumPy z liczbami od 10 do 100 z krokiem 10. Wyciągnij co drugi element tej tablicy, zaczynając od drugiego elementu.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(10, 101, 10)
co_drugi = tablica[1::2]
print("Oryginalna tablica:", tablica)
print("Co drugi element:", co_drugi)
```

Ćwiczenie 20: Wycinanie tablicy wielowymiarowej
Utwórz trójwymiarową tablicę o wymiarach 4x4x3, wypełnioną losowymi wartościami całkowitymi. Wyciągnij drugą warstwę (trzeci wymiar) tej tablicy.

Rozwiązanie:

```
import numpy as np
tablica_3d = np.random.randint(0, 10, size=(4, 4, 3))
druga_warstwa = tablica_3d[:, :, 1]
print(tablica_3d)
```

```
print(druga_warstwa)
```

Ćwiczenie 21: Odwrócenie kolejności elementów
Utwórz tablicę jednowymiarową z liczbami od 1 do 10, a następnie odwróć kolejność elementów w tablicy. Możesz użyć funkcji `flip()`.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(1, 11)
odwrocona = np.flip(tablica)
print("Oryginalna:", tablica)
print("Odwrócona:", odwrocona)
```

Ćwiczenie 22: Modyfikacja fragmentu tablicy
Stwórz dwuwymiarową tablicę 5x5, wypełnioną liczbami od 1 do 25. Zastąp elementy w wierszu 2 i kolumnie 3 wartościami `99`.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(1, 26).reshape(5, 5)
tablica[1, :] = 99  # zmiana wiersza 2
tablica[:, 2] = 99  # zmiana kolumny 3
print(tablica)
```

Ćwiczenie 23: Indeksowanie z warunkiem logicznym
Utwórz tablicę NumPy z liczbami od 1 do 20. Wyciągnij z niej wszystkie liczby parzyste.

Rozwiązanie:

```
import numpy as np
tablica = np.arange(1, 21)
parzyste = tablica[tablica % 2 == 0]
print("Wszystkie liczby:", tablica)
print("Liczby parzyste:", parzyste)
```

Zastanów się, jak będzie wyglądał wynik kiedy tablica będzie dwuwymiarowa.

Ćwiczenie 24: Wyciąganie elementów z użyciem listy indeksów
Stwórz jednowymiarową tablicę zawierającą liczby od 0 do 10. Wyciągnij elementy o indeksach podanych w liście `[1, 3, 5, 7, 9]`.

Rozwiązanie:

```
import numpy as np

tablica = np.arange(11)
wynik = tablica[[1, 3, 5, 7, 9]]
print(wynik)
```

Ćwiczenie 25: Wycinanie tablicy z krokiem
Utwórz tablicę jednowymiarową z liczbami od 1 do 30. Wyciągnij co trzeci element tablicy, zaczynając od piątego elementu.

Rozwiązanie:

```
import numpy as np
```

```
tablica = np.arange(1, 31)
wynik = tablica[4::3]
print(wynik)
```

1.3. Podstawowe operacje na danych w tablicach

Do rozwiązania zadań z tej sekcji wykorzystaj tablice zdefiniowane poniżej:

```
tab_1 = np.array([1, 2, 3, 4, 5, 6])
tab_2 = np.array([5, 6, 7, 8, 9, 10])
tab_3 = np.arange(1, 21)
tab_4 = np.array([[i] for i in range(2, 13, 2)])
```

Zwróć uwagę na wynik wykonywanych operacji arytmetycznych i to jaki sposób są one wykonywane.

Ćwiczenie 26: Operacje arytmetyczne I

1. Wykonaj operację dodawania na tablicach `tab_1` i `tab_2`.

```
tab_1 + tab_2
```

2. Pomnóż każdy element tablicy `tab_1` przez 3.

```
tab_1 * 3
```

3. Oblicz iloczyn elementów `tab_4` z liczbą 0.5.

```
tab_4 * 0.5
```

4. Podziel elementy `tab_2` przez odpowiadające im elementy `tab_1`.

```
tab_2 / tab_1
```

5. Do każdego elementu `tab_3` dodaj wektor `[10, 20, 30, 40, 50]`.

UWAGA Wektor można dodać tylko wtedy gdy wymiary są zgodne, poza tym jest to zwykłe dodawanie macierzy.

```
tab_3.reshape(4,5) + np.array([10, 20, 30, 40, 50])
```

6. Wykonaj dzielenie modulo `tab_2` przez `tab_1`.

```
np.mod(tab_2, tab_1)
```

7. Pomnóż macierz `tab_4` przez wektor `[1, 2, 3]`.

```
tab_4 * np.array([1, 2, 3])
```

8. Podnieś elementy `tab_1` do kwadratu i podziel przez 2.

```
(tab_1 ** 2) / 2
```

9. Wykonaj operację dzielenia całkowitego `tab_3` przez 3.

```
tab_3 // 3
```

10. Pomnóż `tab_4` przez transponowaną wersję samej siebie.

UWAGA Mnożenie macierzy wykonywane jest przez operację `@`, transpozycja to atrybut `.T` dla obiektu `np.ndarray`. Sprawdź co jest wynikiem takiego mnożenia.

```
tab_4 @ tab_4.T
```

Sprawdź także co otrzymasz w wyniku mnożenia tych macierzy w odwrotnej kolejności.

```
tab_4.T @ tab_4
```

1.4. Funkcje uniwersalne

Ćwiczenie 27: `np.sqrt()` i `np.square()`

Utwórz tablicę zawierającą 10 losowych liczb całkowitych z zakresu 1 - 99, a następnie wykorzystaj odpowiednią funkcję do obliczenia:
- pierwiastka kwadratowego każdej liczby,
- kwadratu każdej liczby.

Rozwiązanie:

```
tablica = np.rando.randint(1, 100, 10)
sqrt_tab = np.sqrt(tablica)
square_tab = np.square(tablica)
```

Ćwiczenie 28: `np.abs()`

Oblicz bezwzględne różnice wartości w podanych tablicach:

```
import random
tab_1 = [i + random.random() for i in range(10)]
tab_2 = [i + random.random() for i in range(1,11)]
```

Zanim to zrobisz dokonaj konwersji do tablic NumPy.

Rozwiązanie:

```
tab_1 = np.array(tab_1)
tab_2 = np.array(tab_2)
np.abs(tab_1 - tab_2)
```

Można też wygenerować tablice `tab_` i `tab_2` w nieco inny sposób:

```
tab_1 = np.arange(10) + np.random.random(10)
tab_2 = np.arange(1, 11) + np.random.random(10)
```

Ćwiczenie 29: np.ceil(), np.floor(), np.round()

Wyniki otrzymane w poprzednim zadaniu zaokrąglij w górę i w dół wykorzystując funkcje wymienione w tytule do zadania. Porównaj wynik ich działania.

Rozwiązanie:

```
diff = np.abs(tab_1, tab_2)
print(np.ceil(diff))
print(np.floor(diff))
print(np.round(diff))
```

Ćwiczenie 30: `np.max()` kontra `np.maximum()`

Dane są dwie tablice:

```
tab_1 = np.random.randint(1, 21, 10)
tab_2 = np.random.randint(1, 11, 10)
print(tab_1)
print(tab_2)
```

- Wyznacz maksymalny element każdej z tablic.
- Wybierz większy element z par elementów tych tablic na tych samych pozycjach.

Rozwiązanie:

```
print("wynik działania funkcji max:")
print(np.max(tab_1))
print(np.max(tab_2))
print("Wynik działania funkcji maximum:")
print(np.maximum(tab_1, tab_2))
```

Ćwiczenie 31: Funkcje trygonometryczne

Utwórz tablicę zawierającą kąty w radianach `[π/6, π/4, π/3, π/2, π]`. Wykorzystaj funkcje trygonometryczne do:

- obliczenia sinusa, cosinusa i tangensa każdego kąta,
- sprawdzenia tożsamości trygonometrycznej $\sin^2(x) + \cos^2(x) = 1$,
- znalezienia kątów, dla których `cos(x) > sin(x)`.

Rozwiązanie:

```
katy = np.pi / np.arange(6, 0, -1) # niewątpliwie jest to przykład broadcastingu
sinus = np.sin(katy)
cosinus = np.cos(katy)
tangens = np.tan(katy)
print(np.squaes(sinus) + np.squaes(cosinus))
print(cosinus > sinus)
```

Zwróć uwagę, że warunek logiczny `cosinus > sinus` zwraca listę wartości typu `Bool`. Można ją wykorzystać do filtrowania tablicy.

1.5. Broadcasting

Ćwiczenie 32: Konwersja temperatur
Dokonaj konwersji tabeli temperatur ze skali Celsjusza do skali Fahrenheita.

```
temperatura_C = np.array([
    [20, 22, 21],  # Miasto 1
    [18, 19, 20],  # Miasto 2
    [23, 24, 22],  # Miasto 3
    [17, 18, 19],  # Miasto 4
    [21, 20, 20]   # Miasto 5
])
```

Wzór:

$$T_F = 1.8 \cdot T_C + 32$$

Rozwiązanie:

```
temperatura_F = temperatura_C * 1.8 + 32
print(temperatura_F)
```

Ćwiczenie 33: Normalizacja danych
Oblicz wynik procentowy na podstawie informacji o punktach uzyskanych przez uczniów z testów oraz maksymalnej licznie punktów do uzyskania z każdego testu.

```
wyniki_uczniów = np.array([
    [85, 92, 78],  # Uczeń 1 (matematyka, fizyka, chemia)
    [95, 88, 65],  # Uczeń 2
    [78, 85, 80]   # Uczeń 3
])
punkty_max = np.array([100, 120, 80])
```

Rozwiązanie:

```
wyniki_procentowe = wyniki_uczniów / punkty_max
print(wyniki_uczniów)
```

Ćwiczenie 34: Ceny w sklepie
Oblicz końcowe ceny produktów w poszczególnych dniach na podstawie tabeli cen oraz tabeli rabatów. Rabaty są wspólne dla produktów i naliczane są w kolejne dni.

```
ceny_produktow = np.array([
# produkty
  [100, 70, 140],  # dzień 1
  [120, 80, 150],  # dzień 2
  [120, 80, 130],  # dzień 3
  [130, 70, 120],  # dzień 4
])
# dzień:           1   2   3   4
rabaty = np.array([0, 10, 20, 30])
```

Rozwiązanie: Uwaga na tablicę `rabaty`, do rozwiązania zadnia potrzebna jest tablica 2D z jedną kolumną.

```
cent_po_rabacie = ceny_produktow - rabaty.reshape(-1,1)
```

Ćwiczenie 35: Mapa wartości
Utwórz mapę wartości o rozmiarze 1000x1000 tak, aby w lewym górnym rogu była wartość 0 a prawym dolnym wartość 1. wartości pomiędzy powinny rosnąć liniowo, tak jak ma to miejsce na przykład w tablicy poniżej:

```
[[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1 ]
 [0.   0.02 0.04 0.06 0.08 0.1  0.12 0.14 0.16 0.18 0.2 ]
 [0.   0.03 0.06 0.09 0.12 0.15 0.18 0.21 0.24 0.27 0.3 ]
 [0.   0.04 0.08 0.12 0.16 0.2  0.24 0.28 0.32 0.36 0.4 ]
 [0.   0.05 0.1  0.15 0.2  0.25 0.3  0.35 0.4  0.45 0.5 ]
 [0.   0.06 0.12 0.18 0.24 0.3  0.36 0.42 0.48 0.54 0.6 ]
 [0.   0.07 0.14 0.21 0.28 0.35 0.42 0.49 0.56 0.63 0.7 ]
 [0.   0.08 0.16 0.24 0.32 0.4  0.48 0.56 0.64 0.72 0.8 ]
 [0.   0.09 0.18 0.27 0.36 0.45 0.54 0.63 0.72 0.81 0.9 ]
 [0.   0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1.  ]]
```

Na przecięciu n-tego wiersza i k-tej kolumny znajduje się iloczyn wartości z z pozycji `[n, k-1]` i `[n-1, k]`. Wyświetl fragment tablicy o rozmiarze 5x5 z prawego dolnego rogu.

Rozwiązanie:

```
x = np.linspace(0, 1, 1001).reshape(1, -1)  # tablica 2D z jednym wierszem
y = np.linspace(0, 1, 1001).reshape(-1, 1)  # tablica 2D z jedną kolumną
mapa = x * y
print(mapa[-5:, -5:])
```

Wynik:

```
[[0.992016 0.993012 0.994008 0.995004 0.996   ]
 [0.993012 0.994009 0.995006 0.996003 0.997   ]
 [0.994008 0.995006 0.996004 0.997002 0.998   ]
 [0.995004 0.996003 0.997002 0.998001 0.999   ]
 [0.996    0.997    0.998    0.999    1.      ]]
```

1.6. Własne funkcje uniwersalne

Ćwiczenie 36: Oceny z testów
Napisz funkcję uniwersalną, która przyjmuje tablicę wyników (0-100) i zwraca oceny według skali:
- poniżej 50: „2”
- 50-69: „3”
- 70-84: „4”
- 85-100: „5”

Następnie zastosuj tę funkcję do tablicy 2D reprezentującej wyniki wielu uczniów z różnych testów:

```
wyniki = np.array([
# test:
#     1   2   3   4
    [45, 67, 89, 92],  # uczeń 1
    [78, 83, 95, 89],  # uczeń 2
    [49, 65, 75, 83]   # uczeń 3
])
```

Rozwiązanie:

```
def ocena(x):

    if x < 50:
      return 2
    elif x < 70:
      return 3
    elif x < 85:
      return 4
    else:
      return 5

ocena_vec = np.vectorize(ocena)

print(ocena_vec(wyniki))
```

Ćwiczenie 37: Zmiana wartości
Napisz funkcję uniwersalną, która przyjmuje tablicę danych opisujących czas trwania eksperymentu i dokonuje konwersji z formatu „HH:MM” na sekundy. Przykładowa tablica danych wygląda następująco:

```
experiments_duration = np.array([
  "01:20", "03:15", "02:45", "01:23", "02:24", "01:47"
])
```

Rozwiązanie:

```
def convert(time):
    time = time.split(":")
    return int(time[0]) * 3600 + int(time[1]) * 60
convert_vec = np.vectorize(convert)
print(convert_vec(experiments_duration))
```

Ćwiczenie 38: Tabela BMI
Napisz funkcję, która przyjmuje dwa argumenty: masę wyrażoną w kilogramach oraz wzrost w centymetrach i zwraca wartości:

- `-1` jeśli wartość BMI wskazuje na niedowagę,
- `0` jeśli wartość BMI wskazuje na prawidłową wagę,
- `1` jeśli wartość BMI wskazuje na nadwagę,
- `2` jeśli wartość BMI wskazuje na otyłość. dowolnego stopnia.

Wzór do obliczania BMI:

$$\mathrm{BMI} = \frac{\text{masa w kg}}{(\text{wzrost w m})^2}$$

Parametry BMI:

```
Niedowaga:           BMI poniżej 18,5
Waga prawidłowa:     BMI od 18,5 do 24,9
Nadwaga:             BMI od 25,0 do 29,9
Otyłość I stopnia:   BMI od 30,0 do 34,9
Otyłość II stopnia:  BMI od 35,0 do 39,9
Otyłość III stopnia: BMI 40,0 lub więcej
```

UWAGA Funkcja powinna zwrócić `nan` jeśli wzrost jest równy 0.

Rozwiązanie:

```
def bmi_calc(mass, height):
    if height == 0: return np.nan
    bmi = mass / (height/100)**2
    if bmi ≥ 30: return 2      #  2.0 - patrz opis poniżej
    elif bmi ≥ 25: return 1    #  1.0
    elif bmi ≥ 18.5: return 0  #  0.0
    else: return -1            # -1.0

bmi_calc_vec = np.vectorize(bmi_calc)

mass = np.array([90, 80, 94, 107])
height = np.array([172, 189, 0, 164])

print(bmi_calc_vec(mass, height))
```

To rozwiązanie wygeneruje błąd w sytuacji kiedy wzrost będzie równy zero. Jest to spowodowane brakiem możliwości konwersji `nan` na typ `int`. Aby ten błąd naprawić należy zwracać nie wartości całkowite ale rzeczywiste: `-1.0`, `0.0`, itd.

1.7. Funkcje agregujące i statystyki opisowe

Zanim przystąpimy do rozwiązywania zadań napiszmy sobie trzy funkcje do generowania zestawów danych.

1. Napisz funkcję, która zwraca tablicę o rozmiarze `n` wypełnioną liczbami całkowitymi z podanego zakresu. Rozkład wartości powinien być jednostajny.

```
import numpy as np

def tablica_jednostajna(n, min_wartosc, max_wartosc):

    tablica = np.random.randint(min_wartosc, max_wartosc + 1, size=n)
    return tablica
```

2. Zmodyfikuj funkcję, z poprzedniego zadania,w taki sposób, aby zwracała liczby całkowite spełniające wymogi rozkładu normalnego. Liczby powinny być skupione wokół podanej wartości z ustalonym rozrzutem.

```
import numpy as np

def tablica_normalna(n, srednia, odchylenie_standardowe):

    tablica = np.random.normal(loc=srednia, scale=odchylenie_standardowe, size=n)
    tablica = np.round(tablica).astype(int)
    return tablica
```

3. Napisz funkcję, która zwraca tablice o rozmiarze `n x m` wypełnioną wartościami opisanymi w punkcie 1.

```
def tablica_jednostajna_nm(n, m, min_wartosc, max_wartosc):

    return tablica_jednostajna(n * m, min_wartosc, max_wartosc).reshape(n, m)
```

Ćwiczenie 39: Temperatury
Masz tablicę pomiarów temperatury z 24 godzin:

```
temperatury = tablica_jednostajna(24, 18, 26)
```

Napisz program, który:
- znajdzie najwyższą i najniższą temperaturę,
- obliczy średnią temperaturę,
- obliczy różnicę między temperaturą maksymalną a minimalną.
- obliczy liczbę dni ze średnią temperaturą większą niż 22 stopnie,
- obliczy średnią temperaturę dla zbioru temperatur powyżej 22 stopni.

Rozwiązanie:

```
# Najwyższa i najniższa temperatura
temp_max = np.max(temperatury)
temp_min = np.min(temperatury)

# Średnia temperatura
temp_srednia = np.mean(temperatury)

# Różnica między temperaturą maksymalną a minimalną
roznica = temp_max - temp_min

# Liczba dni ze średnią temperaturą > 22 stopnie
```

```
liczba_cieplych = np.sum(temperatury > 22)

# Średnia temperatura dla zbioru temperatur powyżej 22 stopni
temp_wysokie = temperatury[temperatury > 22]
srednia_wysokich = np.mean(temp_wysokie)
```

Ćwiczenie 40: Zawody sportowe
Dana jest tablica wyników trzech drużyn w czterech konkurencjach:

```
wyniki = tablica_jednostajna_nm(3, 4, 0, 10)
```

Wyniki każdej z drużyn mieszczą się w przedziale od 0 do 10. Napisz program, który:
- obliczy średni wynik każdej drużyny,
- znajdzie najlepszy wynik w każdej konkurencji,
- wskaże, która drużyna miała najmniejsze odchylenie standardowe wyników.

Rozwiązanie:

Do wypisywania zagregowanych elementów tablic (będących też tablicami) wykorzystamy funkcję `enumaerate()` zwracająca krotkę `(numer porządkowy, element z argumentu funkcji enumerate)`

```
# Średni wynik każdej drużyny (axis=1 bo liczymy wzdłuż wierszy)
srednie_druzyn = np.mean(wyniki, axis=1)
# Warto wypisać te średnie:
for i, srednia in enumerate(srednie_druzyn):
    print(f"Średnia drużyny {i+1}: {srednia:.2f}")

# Najlepszy wynik w każdej konkurencji (axis=0 bo liczymy wzdłuż kolumn)
najlepsze_wyniki = np.max(wyniki, axis=0)
# Najlepsze wyniki wypisujemy, podobnie jak poprzednio:
for i, wynik in enumerate(najlepsze_wyniki):
    print(f"Najlepszy wynik w konkurencji {i+1}: {wynik:.2f}")

# Odchylenie standardowe dla każdej drużyny
odchylenia = np.std(wyniki, axis=1)
# Znalezienie indeksu drużyny z najmniejszym odchyleniem
# Funkcja argmin zwraca indeks najmniejszego elementu tablicy
druzyna_minstd_index = np.argmin(odchylenia)
print(f"Najmniejsze odchylenie standardowe ma drużyna {druzyna_minstd_index + 1}")
```

Ćwiczenie 41: Analiza sprzedaży
Masz dane sprzedaży z pięciu sklepów za ostatni tydzień:

```
sprzedaz = tablica_jednostajna_nm(5, 7, 10, 20) * 1000
```

Napisz program, który:
- obliczy łączną sprzedaż każdego sklepu,
- znajdzie dzień z najwyższą łączną sprzedażą we wszystkich sklepach,
- obliczy średnią dzienną sprzedaż dla każdego sklepu.

Rozwiązanie:

Rezygnujemy z wypisywania poszczególnych wyników, można to zrobić podobnie jak w zadaniu powyżej.

```
# Łączna sprzedaż każdego sklepu (sumujemy wzdłuż wierszy - axis=1)
laczna_sprzedaz = np.sum(sprzedaz, axis=1)
```

```
# Znajdź dzień z najwyższą łączną sprzedażą
# Najpierw sumujemy sprzedaż wszystkich sklepów dla każdego dnia (axis=0)
# najlepsza_dzienna przechowuje indeks dnia z najlepszym utargiem
sprzedaz_dzienna = np.sum(sprzedaz, axis=0)
najlepszy_dzien = np.argmax(sprzedaz_dzienna)

# Średnia dzienna sprzedaż dla każdego sklepu (axis=1)
srednia_dzienna = np.mean(sprzedaz, axis=1)
```

Ćwiczenie 42: Problemy z ciśnieniem
Masz tablicę z pomiarami ciśnienia krwi pacjentów:

```
cisnienie = np.array([
# [skurczowe, rozkurczowe]
    [120, 80],  # pacjent 1
    [130, 85],  # pacjent 2
    [125, 82],  # ...
    [140, 90],
    [135, 88],
    [128, 83]
])
```

Napisz program, który znajdzie:
- indeksy pacjentów z ciśnieniem skurczowym powyżej 130
- liczbę pacjentów z ciśnieniem rozkurczowym poniżej 85
- średnia wartość ciśnienia dla pacjentów z prawidłowym ciśnieniem (`skurczowe ≤ 130` i `rozkurczowe ≤ 85`)

Rozwiązanie:

Zwróćmy uwagę, na fakt, iż `cisnienie` to tablica o dwóch wymiarach: wiesze to pacjenci, kolumny to ciśnienie skurczowe i rozkurczowe. Możemy tu wykorzystać funkcję `np.where()`. Zwróci ona krotkę indeksów pozycji, gdzie spełniony jest warunek:

```
# Indeksy pacjentów z ciśnieniem skurczowym > 130
wysokie_skurczowe = np.where(cisnienie[:, 0] > 130)[0]

# uwaga - np.where zwraca krotkę indeksów, dla tablicy 1D
# ta krotka zawiera tylko jeden element dlatego na końcu
# pojawia się odniesienie do tego elementu w postaci indeksu [0]
# aby wypisać ciśnienie dla tych pacjentów korzystamy z wywołania:
print(cisnienie[wyokie_skurczowe])

# Liczba pacjentów z ciśnieniem rozkurczowym < 85
niskie_rozkurczowe = np.sum(cisnienie[:, 1] < 85)

# Maska dla pacjentów z prawidłowym ciśnieniem
maska_prawidlowe = (cisnienie[:, 0] ≤ 130) & (cisnienie[:, 1] ≤ 85)
prawidlowe_cisnienie = cisnienie[maska_prawidlowe]

# wyliczamy średnie ciśnienie z danych pozostałych po nałożeniu maski
# średnia wyliczana jest wzdłuż odpowiedniej osi
srednie_prawidlowe = np.mean(prawidlowe_cisnienie, axis=0)
```

Ćwiczenie 43: Analiza ocen
Masz tablicę wyników procentowych z testu dla grupy studentów:

```
wyniki = tablica_normalna(100, 65, 30)
# zadbajmy o to aby wyniki studentów miały sens:
wyniki[wynii < 0] = 0
wyniki[wyniki >100] = 100
```

Możemy tu też użyć funkcji `np.clip()`:

```
wyniki = np.clip(tablica_normalna(100, 65, 30), 0, 100)
```

która ogranicza zakres danych do wskazanego przedziału.

Napisz program który:
- używając `np.where()`, przypisze oceny według schematu:
  - 90 i więcej: 5.0
  - 80-89: 4.5
  - 70-79: 4.0
  - 60-69: 3.5
  - 50-59: 3.0
  - poniżej 50: 2.0
- wyświetli pierwotny wynik obok oceny dla każdego studenta,
- policzy, ilu studentów otrzymało każdą z ocen.

Przykład wyniku:

```
Wynik 45% → ocena 2.0
Wynik 67% → ocena 3.5
...
```

oraz

```
Studenci z oceną 5.0: 12
Studenci z oceną 4.5: 7
...
```

Rozwiązanie:

Zadanie wymaga zastosowania funkcji `np.where()` w klasycznym przypadku.

```
# Przypisanie ocen używając np.where()
oceny = np.where(wyniki ≥ 90, 5.0,
       np.where(wyniki ≥ 80, 4.5,
       np.where(wyniki ≥ 70, 4.0,
       np.where(wyniki ≥ 60, 3.5,
       np.where(wyniki ≥ 50, 3.0, 2.0)))))

# Wyświetlenie wyników i ocen
for wynik, ocena in zip(wyniki, oceny):
   print(f"Wynik {wynik:.1f}% → ocena {ocena}")

# Policzenie liczby studentów z każdą oceną
oceny_wartosci = [2.0, 3.0, 3.5, 4.0, 4.5, 5.0]

for ocena in oceny_wartosci:
```

```
    liczba_studentow = np.sum(oceny == ocena)
    print(f"Studenci z oceną {ocena}: {liczba_studentow}")
```

1.8. Łączenie i dzielenie tablic

Proste (wizualne) wyjaśnienie koncepcji wymiarów w NuumPy: https://jalammar.github.io/visual-numpy/

Ćwiczenie 44: Jak działa `stack`?
Mamy trzy tablice:

```
tab_1 = np.ones((4, 4))
tab_2 = tab_1 * 2
tab_3 = tab_1 * 3
```

Spróbuj narysować sobie jak będą wyglądały następujące tablice:

```
stack_1 = np.stack((tab_1, tab_2, tab_3), axis=2)
stack_2 = np.stack((tab_1, tab_2, tab_3), axis=0)
```

Wyjaśnij sens wywołań:

```
print(stack_1[0,0])
print(stack_2[0,0])
```

Ćwiczenie 45: `np.concatenate()`
Wykorzystując tablice `tab_1` i `tab_2` z poprzedniego zadania utwórz tablicę pokazaną poniżej:

```
[[1. 1. 1. 1. 2. 2. 2. 2.]
 [1. 1. 1. 1. 2. 2. 2. 2.]
 [1. 1. 1. 1. 2. 2. 2. 2.]
 [1. 1. 1. 1. 2. 2. 2. 2.]
 [2. 2. 2. 2. 1. 1. 1. 1.]
 [2. 2. 2. 2. 1. 1. 1. 1.]
 [2. 2. 2. 2. 1. 1. 1. 1.]
 [2. 2. 2. 2. 1. 1. 1. 1.]]
```

Rozwiązanie:

```
t1 = np.concatenate((tab_1, tab_2), axis = 0)
t2 = np.concatenate((tab_2, tab_1), axis = 0)
t3 = np.concatenate((t1, t2), axis = 1)
```

Ćwiczenie 46: Inny sposób
Utwórz tablicę opisana w poprzednim zadaniu wykorzystując zaawansowane metody selekcji wycinków.

Rozwiązanie:

```
n = 8
tab = np.ones((n, n))
tab[n//2:n, 0:n//2] = tab[n//2:n, 0:n//2] * 2
tab[0:n//2, n//2:n] = tab[0:n//2, n//2:n] * 2
print(tab)
```

Ćwiczenie 47: Uśrednianie pikseli
Tablice zdefiniowane poniżej opisują trzy obrazy o rozmiarze 16x16 pikseli w skali szarości o wartościach z przedziału od 0 do 1. Dokonaj „złożenia" tych obrazów w jeden, uśredniając wartości dla każdego piksela.

```
n = 16
p1 = np.random.rand(n**2).reshape(n, n)
```

```
p2 = np.random.rand(n**2).reshape(n, n)
p3 = np.random.rand(n**2).reshape(n, n)
```

Rozwiązanie:

```
# złożenie obrazów wzdłuż osi 0
p = np.stack((p1, p2, p3), axis=0)
# obliczenie średniej wzdłuż nowej osi - uśrednianie są "szarości" dla każdego piksela
mean_p = np.mean(p, axis=0)
```

Zwróć uwagę, na fakt, iż `p[:, 0, 0]` zwraca tablicę trójelementową zawierającą „zestackowane" wartości szarości dla piksela `[0, 0]`. Uśrednianie obywa się wzdłuż osi 0, co oznacza, że uśrednione zostaną te właśnie wartości. Po uśrednianiu wynikiem będzie tablica o tylko dwóch wymiarach. Uśrednianie możemy też zrobić w inny sposób:

```
p_mean = (p1 + p2 + p3) / 3
```

Ćwiczenie 48: Nieśmiertelne zadanie z BMI
Dysponujemy trzema tablicami zawierającymi dane dotyczące pięciu pacjentów:

```
wiek = tablica_jednostajna_nm(5, 1, 20, 80)  # lata
waga = tablica_jednostajna_nm(5, 1, 50, 100) # kilogramy
wzrost = tablica_jednostajna_nm(5, 1, 150, 190)  # centymetry
```

Połącz te w jedną tablicę gdzie każdy wiersz reprezentuje jednego pacjenta. Następnie oblicz wskaźnik BMI dla każdego pacjenta. Obliczone BMI dodaj jako nową kolumnę.

Rozwiązanie:

```
# Łączenie danych w jedną tablicę
dane_pacjentow = np.hstack((wiek, waga, wzrost))

# Obliczanie BMI (wzrost musimy zamienić na metry)
wzrost_w_metrach = wzrost / 100
bmi = waga / (wzrost_w_metrach ** 2)

# Dodanie BMI jako nowej kolumny
dane_z_bmi = np.hstack((dane_pacjentow, bmi))
```

Ćwiczenie 49: Roczne wyniki sprzedaży
Masz tablicę zawierającą wyniki sprzedaży z całego roku (365 dni):

```
sprzedaz = np.random.uniform(100, 200, size=365)
```

Podzieli tablicę na 12 części odpowiadających miesiącom (zakładamy dla uproszczenia, że każdy miesiąc ma równą liczbę dni) i oblicz średnią sprzedaż dla każdego miesiąca. Wskaż miesiąc z najlepszą sprzedażą.

Rozwiązanie:

```
miesiace = np.array_split(sprzedaz, 12)
miesiace_srednia = np.mean(miesiace)
```

Proszę zwrócić uwagę na liczbę dni w poszczególnych miesiącach. Jest to konsekwencja stosowania `np.array_split()`.

1.9. Rozszerzanie i powtarzanie tablic

Ćwiczenie 50: Szachownica
Utwórz wzór szachownicy 2x2:

```
wzor = np.array([[1, 0],
                 [0, 1]])
```

Napisz program, który powieli ten wzór 4 razy w pionie i w poziomie używając `np.tile()`, wyświetli powstałą tablicę 8x8 i policzy sumę elementów w każdym rogu powstałej tablicy (obszary 2x2).

Rozwiązanie:

```
# Powielenie wzoru 4x4 używając np.tile()
szachownica = np.tile(wzor, (4, 4))
print(szachownica)

# Policzenie sum w rogach
# Lewy górny róg
suma_lg = np.sum(szachownica[0:2, 0:2])
# Prawy górny róg
suma_pg = np.sum(szachownica[0:2, -2:])
# Lewy dolny róg
suma_ld = np.sum(szachownica[-2:, 0:2])
# Prawy dolny róg
suma_pd = np.sum(szachownica[-2:, -2:])
```

Ćwiczenie 51: Dane pomiarowe
Mamy jednowymiarową tablicę zawierającą dane:

```
dane = np.array([56, 75, 69, 55, 49, 59, 77, 82])
```

Napisz program, który, doda kolejny wymiar do tablicy, utworzy tablicę z trzema kolejnymi danymi dla każdej wartości używając `np.repeat()` i doda losowy szum ± 2 do powtórzonych danych.

Rozwiązanie:

```
# Rozszerzenie tablicy do kształtu (8,1)
dane_2d = dane[:, np.newaxis]

# Utworzenie 3 pomiarów dla każdej wartości z danych
# Możemy to zrobić na dwa sposoby:
# Sposób 1: użycie repeat
dane_repeat = np.repeat(tdane_2d, 3, axis=1)

# Sposób 2: użycie tile
dane_tile = np.tile(dane_2d, (1, 3))

# Dodanie losowego szumu ±2
szum = np.random.uniform(-2, 2, dane_tile.shape)
dane_szum = dane_tile + szum

print(dane_szum)
```

Ćwiczenie 52: Siatka punktów
Mając tablicę wartości x:

```
x = np.array([1, 2, 3, 4, 5])
```

Napisz program, który utworzy siatkę 5x5 wartości, gdzie każdy wiersz to wartości x, utworzy drugą siatkę 5x5, gdzie każda kolumna to wartości x. Następnie, używając tych siatek, oblicz wartości funkcji $f(x, y) = x \cdot y$ dla wszystkich punktów siatki.

Rozwiązanie:

```
# każdy wiersz to wartości x
siatka_x = np.tile(x, (5, 1))
# każda kolumna to wartości x
# należy na wstępie przekształcić wiersz an kolumnę
siatka_y = np.tile(x[:, np.newaxis], (1, 5))

wartosci_funkcji = siatka_x * siatka_y
```

Zauważmy, że z uwagi na prostotę funkcji, która mamy wyliczyć możemy zrobić to znacznie prościej, wykorzystując broadcasting:

```
# wiersz x kolumna
wartosci_funkcji = x * x[:, np.newaxis]
```

Ostatecznie możemy też skorzystać z funkcji `meshgrid()`:

```
X, Y = np.meshgrid(x, x)
wartosci_funkicji = X * Y
```

Ćwiczenie 53: Obrazy dla modelu ML
Mamy zestaw obrazów w skali szarości, zdefiniowanych następująco:

```
obrazy = np.random.rand(10, 32, 32)  # 10 obrazów 32x32 piksele
```

Napisz program, który doda wymiar kanału koloru do każdego obrazu (zmieniając kształt na (10,28,28,1)), stworzy kopię każdego obrazu z obrotem o 90 stopni i połączy oryginalne i obrócone obrazy w jeden zestaw danych (łączenie względem osi 0).

Rozwiązanie:

```
# Dodanie wymiaru kanału koloru
# Zmiana kształtu z (10, 28, 28) na (10, 28, 28, 1)
obrazy_z_kanalem = np.expand_dims(obrazy, axis=3)

# Tworzenie kopii z obrotem o 90 stopni
# np.rot90 obraca tablicę o 90 stopni przeciwnie do ruchu wskazówek zegara
# k = 1 - ile razy obrócić
# axes = (1, 2) to są osie dla obrazu
obrazy_obrocone = np.rot90(obrazy_z_kanalem, k=1, axes=(1, 2))

# Połączenie oryginalnych i obróconych obrazów
# Łączymy wzdłuż pierwszej osi (zwiększamy liczbę obrazów)
wszystkie_obrazy = np.concatenate((obrazy_z_kanalem, obrazy_obrocone), axis=0)
```

Zadania sprawdzające

Zadanie 1

Dana jest tablica:

```
tablica = np.arange(100).reshape(10,10)
```

Napisz funkcję, która jako argumenty przyjmuje tę tablicę i krotkę indeksów `(i, j, k, l)` i dla podanych indeksów zwraca podtablicę tablicy. W przypadku błędnych wartości zwracany jest obiekt `None`. Sprawdź, czy przekroczenie zakresu indeksów wywołuje błąd w NumPy.

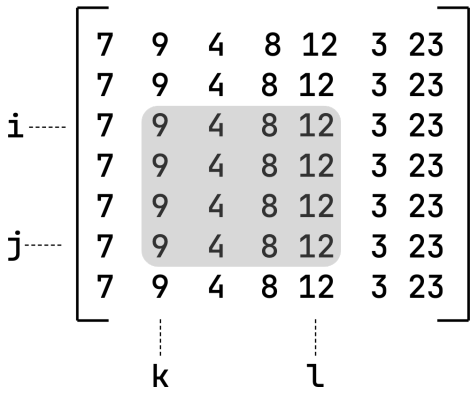


Figure 1: Rysunek Do zadnia 1.

Zadanie 2

Poniższy fragment kodu generuje obrazek przedstawiający "zawartość" tablicy $n \times n$ w postaci mapy kolorów:

```
import numpy as np
import matplotlib.pyplot as plt

array = np.random.rand(50,50)

plt.imshow(array)
plt.show()
```

Wykorzystując ten kod napisz program, który generuje mapę kolorów odzwierciedlających wartości funkcji:

$$f(x, y) = \sin(x) \cdot \cos(y)$$

w przedziale od $-\pi$ do π. Obrazek powinien mieć rozmiar 100×100 punktów.

Zadanie 3
Poniższy kod generuje tablicę punktów uzyskanych przez poszczególnych uczniów (wiersze) ze sprawdzianu składającego się z 10 zadań.

```
import numpy as np

wyniki = np.random.randint(0,10, 100).reshape(10,10)
```

Maksymalna ilość punktów możliwa do zdobycia przez ucznia wynosiła 100. Napisz funkcję, która przyjmuje tablicę wyników i zwraca kolumnę ocen jakie uzyskali uczniowie. Skala ocen jest nastepująca:

```
 0 -  39: 1.0
40 -  49: 1.5
50 -  55: 2.0
55 -  59: 2.5
60 -  69: 3.0
70 -  74: 3.5
75 -  79: 4.0
80 -  89: 4.5
90 - 100: 5.0
```

Oblicz wektor zawierający, średnią ilość punktów, jaką uczniowie uzyskali za poszczególne zadania (nie musi być zwracany przez funkcję).

Zadanie 4
Napisz funkcję, która znajduje rozwiązanie układu równań liniowych:

$$
\begin{aligned}
&a_{1,1}x_1 + a_{1,2}x_2 + ...a_{1,n}x_n = c_1 \\
&a_{2,1}x_1 + a_{2,2}x_2 + ...a_{2,n}x_n = c_2 \\
&... \\
&a_{n,1}x_1 + a_{n,2}x_2 + ...a_{n,n}x_n = c_n
\end{aligned}
$$

Argumentem funkcji jest tablica współczynników postaci:

$$
\begin{bmatrix}
a_{1,1} & a_{1,2} & \dots & a_{1,n} & b_1 \\
a_{2,1} & a_{2,2} & \dots & a_{2,n} & b_2 \\
\vdots & \vdots & \ddots & \vdots & \vdots \\
a_{n,1} & a_{n,2} & \dots & a_{n,n} & b_n
\end{bmatrix}
$$

Funkcja zwraca:

- tablicę kolumnową z rozwiązaniami w przypadku układu oznaczonego,
- wartość None w przypadku układu nieoznaczonego,
- pustą tablicę w przypadku układu sprzecznego.

Możesz wykorzystać funkcję `np.linalg.det()`, która wylicza wyznacznik macierzy kwadratowej.

Zadanie 5
Wykorzystując informacje zawarte w zadaniu 2 napisz program, który generuje obrazek przedstawiający zbiór Mandelbrota. Siatka punktów powinna obejmować zakresy:

- od –2 do 1 na osi x ($Re(z)$)
- od –1.5 do 1.5 na osi y ($Im(z)$)

z gęstością co 0.01 - co oznacza 100 punktów na oś.